NAME - KRISH CHANDRASHEKHAR PACHODE ROLL NO.- SY-B-18 BATCH - AIDS(SY-B)

THEORY -

1. What is data ingestion and why is it important in data science? Which Python library is commonly used for data ingestion and cleaning?** Data ingestion is the process of collecting and importing data from various sources such as CSV files, databases, or APIs into a system for analysis. It is important because it is the first step in data science and ensures that data is available for processing and analysis. The most commonly used Python library for data ingestion and cleaning is **Pandas**.
2. Why is data cleaning necessary before performing analysis or machine learning?** Data cleaning is necessary because real-world data often contains missing values, errors, duplicates, and inconsistencies. Cleaning the data improves accuracy, ensures reliable results, and helps in building better machine learning models. Without cleaning, the analysis may produce incorrect results.
3. How do you load a CSV file into a Pandas DataFrame?** A CSV file can be loaded into a Pandas DataFrame using the `read_csv()` function.

Example:

```
import pandas as pd
df = pd.read_csv("data.csv")
```

4. What is the difference between `head()` and `tail()` functions in Pandas?** The `head()` function displays the first five rows of a dataset, while the `tail()` function displays the last five rows. Both are used to quickly inspect the data.

Example:

```
df.head()
df.tail()
```

---

5. How can you identify missing values in a dataset?** Missing values in a dataset can be identified using the `isnull()` function in Pandas. To count missing values in each column, `sum()` is used.

Example:

```
df.isnull()
df.isnull().sum()
```

```
import pandas as pd
import numpy as np

# 1. Load a CSV file using pandas and display first five rows
print("Q1: Load a CSV file using pandas and display first five rows")
df1 = pd.DataFrame({"A": [1,2,3,4,5,6], "B": [10,20,30,40,50,60]})
print(df1.head())
print()

# 2. Identify missing values in a dataset
print("Q2: Identify missing values in a dataset")
df2 = pd.DataFrame({"Name": ["Aman", None, "Riya"], "Marks": [90, 85, None]})
print(df2.isnull().sum())
print()

# 3. Replace missing values with zero
print("Q3: Replace missing values with zero")
df3 = pd.DataFrame({"X": [1, None, 3], "Y": [None, 5, 6]})
print(df3.fillna(0))
print()

# 4. Rename column names to lowercase
print("Q4: Rename column names to lowercase")
df4 = pd.DataFrame({"Name": ["A"], "AGE": [20]})
df4.columns = df4.columns.str.lower()
print(df4.columns)
print()

# 5. Display dataset shape (rows and columns)
print("Q5: Display dataset shape")
df5 = pd.DataFrame({"A":[1,2,3], "B":[4,5,6]})
print(df5.shape)
print()

# 6. Remove duplicate rows from a dataset
print("Q6: Remove duplicate rows")
df6 = pd.DataFrame({"A":[1,1,2,3], "B": [4,4,5,6]})
print(df6.drop_duplicates())
print()

# 7. Handle missing values using mean or median
print("Q7: Handle missing values using mean")
df7 = pd.DataFrame({"Marks":[90, None, 80, None]})
df7["Marks"] = df7["Marks"].fillna(df7["Marks"].mean())
print(df7)
```

```
print()

# 8. Change data types of selected columns
print("Q8: Change data types")
df8 = pd.DataFrame({"A":[1,2,3]})
df8["A"] = df8["A"].astype(float)
print(df8.dtypes)
print()

# 9. Filter rows based on a given condition
print("Q9: Filter rows where value > 50")
df9 = pd.DataFrame({"Marks":[40, 60, 75, 30]})
print(df9[df9["Marks"] > 50])
print()

# 10. Save the cleaned dataset to a new CSV file
print("Q10: Save dataset to CSV")
df10 = pd.DataFrame({"A":[1,2,3]})
df10.to_csv("output.csv", index=False)
print("File saved")
print()

# 11. Compare different missing value handling strategies
print("Q11: Compare mean and median")
df11 = pd.DataFrame({"Marks":[10, None, 30, 100]})
mean_filled = df11.fillna(df11.mean())
median_filled = df11.fillna(df11.median())
print("Mean:\n", mean_filled)
print("Median:\n", median_filled)
print()

# 12. Clean inconsistent categorical values
print("Q12: Clean categorical values")
df12 = pd.DataFrame({"City":["Pune ", "pune", "MUMBAI"]})
df12["City"] = df12["City"].str.lower().str.strip()
print(df12)
print()

# 13. Automate data cleaning steps using functions
print("Q13: Automate cleaning")
def clean(df):
    return df.drop_duplicates().fillna(0)

df13 = pd.DataFrame({"A":[1,1,None]})
print(clean(df13))
print()

# 14. Perform outlier detection
print("Q14: Outlier detection")
df14 = pd.DataFrame({"Marks":[10, 12, 14, 100]})
Q1 = df14["Marks"].quantile(0.25)
```

```
Q3 = df14["Marks"].quantile(0.75)
IQR = Q3 - Q1
outliers = df14[(df14["Marks"] < Q1 - 1.5*IQR) | (df14["Marks"] > Q3 + 1.5*IQR)
print(outliers)
print()

# 15. Generate data cleaning report
print("Q15: Data cleaning report")
df15 = pd.DataFrame({"A":[1,1,None]})
cleaned = df15.drop_duplicates().fillna(0)
print("Original shape:", df15.shape)
print("Cleaned shape:", cleaned.shape)
print("Missing values:\n", cleaned.isnull().sum())
```

```
0  1  4
2  2  5
3  3  6

Q7: Handle missing values using mean
   Marks
0   90.0
1   85.0
2   80.0
3   85.0

Q8: Change data types
A    float64
```

```
1    pune
2  mumbai

Q13: Automate cleaning
     A
0  1.0
2  0.0

Q14: Outlier detection
   Marks
3    100

Q15: Data cleaning report
Original shape: (3, 1)
Cleaned shape: (2, 1)
Missing values:
 A    0
dtype: int64
```

Conclusion –

In this experiment, we learned how to perform data ingestion and data cleaning using Pandas. We practiced loading data, identifying and handling missing values, removing duplicates, changing data types, and filtering data. We also applied advanced techniques like outlier detection, handling inconsistent data, and automating cleaning using functions.

Overall, this experiment helped in understanding how to prepare raw data for analysis, which is an essential step in data science to ensure accurate and reliable results.